

Task 03 - Question Answers

C# Fundamentals - Parsing, Objects, References, Strings & StringBuilder

Q1. What is the difference between `int.Parse` and `Convert.ToInt32` when handling null inputs?

`int.Parse(null)` throws an **ArgumentNullException** - it does not accept a null string at all.

`Convert.ToInt32(null)`, on the other hand, does **not** throw an exception for a null string; it simply returns **0**. This makes `Convert.ToInt32` slightly more forgiving when the source value might be null, while `int.Parse` expects a valid, non-null numeric string and will fail loudly if that expectation isn't met.

Q2. Why is `TryParse` recommended over `Parse` in user-facing applications?

`TryParse` is recommended because it avoids throwing exceptions when the input is invalid. Instead of crashing the program or requiring a try-catch block, it returns a **bool** indicating success or failure and outputs the converted value through an out parameter (0 if conversion failed). Since user input is unpredictable and exceptions are relatively expensive to throw and catch, `TryParse` leads to cleaner, safer, and better-performing validation logic when dealing with input coming directly from a user, such as a text box or console prompt.

Q3. Explain the real purpose of the `GetHashCode()` method.

`GetHashCode()` returns an integer that represents a "hash code" for an object, primarily used to support efficient storage and lookup in hash-based collections such as `Dictionary<TKey,TValue>` and `HashSet<T>`. Objects that are considered equal (via `Equals()`) must return the same hash code, so that the collection can quickly narrow down where to look for (or place) an item without comparing it against every other item. It is **not** meant to generate a unique ID for an object, and it is not guaranteed to stay the same across different runs of a program or different versions of .NET - its only real contract is consistency with `Equals()` and reasonably even distribution for performance.

Q4. What is the significance of reference equality in .NET?

Reference equality means that two variables refer to the exact same object in memory, not just objects that happen to have equal field values. In .NET, `ReferenceEquals(a, b)` (or the default `==` behavior for reference types that don't override it) checks this directly. This matters because with reference types, modifying the object through one variable is visible through every other variable that references the same object - which is exactly why understanding reference equality is essential for predicting how changes propagate, avoiding unintended side effects, and correctly reasoning about mutable shared state in an application.

Q5. Why is string immutable in C#?

A string in C# cannot be changed after it is created - any operation that appears to "modify" a string (like concatenation) actually creates a brand-new string object in memory and leaves the original untouched. This is by design, for several reasons: it makes strings inherently thread-safe (multiple

threads can read the same string with no risk of one thread changing it under another's feet), it allows the runtime to safely cache and reuse identical string literals (string interning) to save memory, and it prevents bugs where code holding a reference to a string is surprised by it changing unexpectedly elsewhere in the program.

Q6. How does `StringBuilder` address the inefficiencies of string concatenation?

Because strings are immutable, repeatedly concatenating strings (e.g., in a loop) creates a new string object on every single concatenation, copying all the previous characters each time - this becomes very expensive and memory-intensive as the number of concatenations grows. `StringBuilder` solves this by maintaining an internal, resizable **mutable character buffer**. Operations like `Append()`, `Insert()`, `Replace()`, and `Remove()` modify that buffer directly, in place, without creating a new object each time - only when the buffer's capacity is exceeded does it resize (typically by doubling), which happens far less often than a new allocation on every change.

Q7. Why is `StringBuilder` faster for large-scale string modifications?

For large-scale or repeated modifications, `StringBuilder` avoids the repeated allocation and copying that plain string concatenation requires. A loop that concatenates strings n times using `+` has a cost that grows roughly with the square of n ($O(n^2)$), since each step copies everything built so far into a new string. `StringBuilder` instead grows its internal buffer only occasionally and writes new data directly into existing memory, giving it roughly linear ($O(n)$) performance overall. The larger the number of modifications, the bigger this performance gap becomes.

Q8. Which string formatting method is most used, and why?

In modern C# code, **string interpolation** (using the `$` prefix, e.g., `$"Sum is {sum}"`) is generally the most used and recommended method. It is more readable than `string.Format` because the variables appear directly inside the string rather than as separate indexed placeholders, it's checked by the compiler at compile time (catching typos in variable names), and it's just as flexible for formatting values (e.g., `$"{price:C}"`). Plain concatenation with `+` is still common for very simple cases, but it becomes harder to read and less efficient as more values are combined, and `string.Format` is now mostly seen in older codebases or scenarios needing a format string built dynamically at runtime.

Q9. Explain how `StringBuilder` is designed to handle frequent modifications compared to strings.

Unlike `string`, which is immutable and creates a new object for every change, `StringBuilder` is built around a **mutable internal buffer** that supports in-place edits. Methods such as `Append()`, `Insert()`, `Replace()`, and `Remove()` directly modify that buffer's contents rather than allocating a new object each time. `StringBuilder` also over-allocates its buffer capacity so it can absorb several small changes without needing to resize on every single one, only expanding (typically doubling) once the current capacity is exceeded. This design makes it the appropriate tool whenever a string is going to be built or changed frequently, such as inside a loop, while plain strings remain simpler and perfectly fine for values that are set once and rarely, if ever, modified.